



HCMUS

NFS – Network File Sharing Between Linux Servers

Architecture, Production Implementation, and Automated Persistency

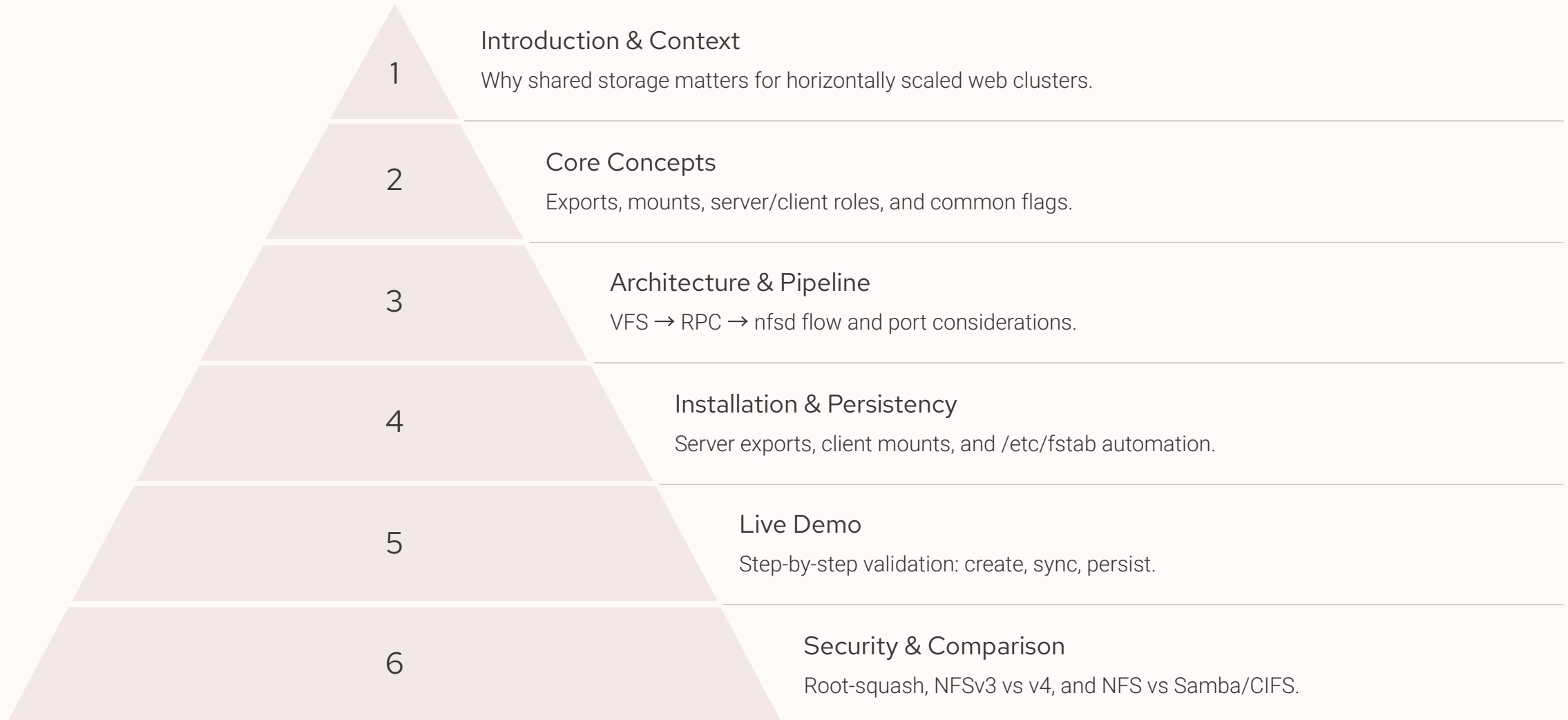
VNU-HCM University of Science (HCMUS)

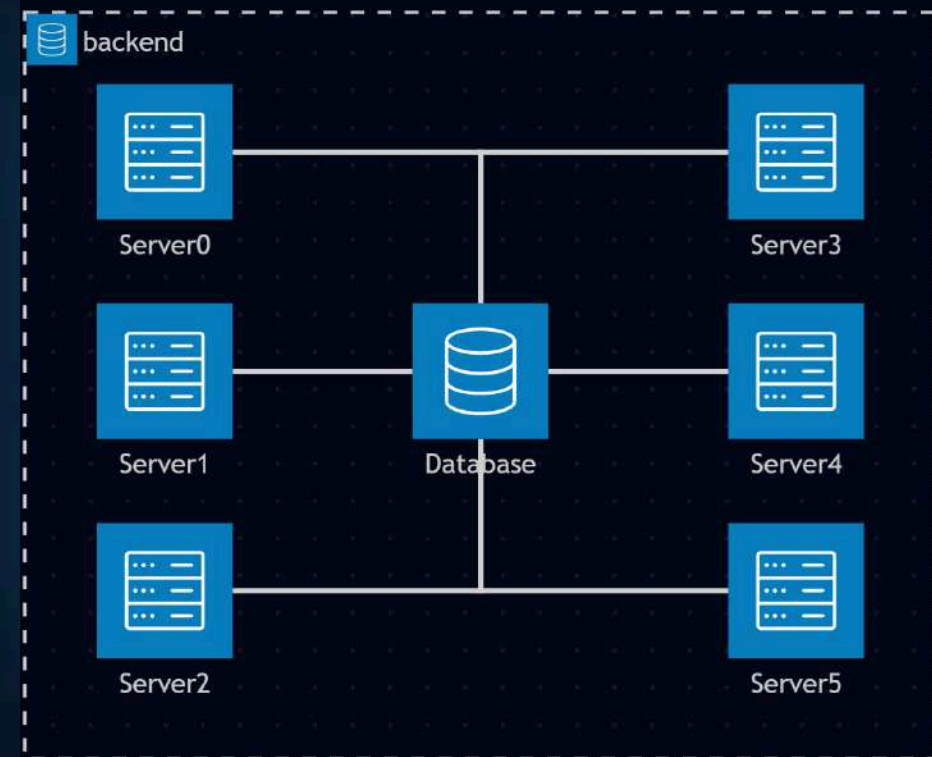


Group Members:

- Đặng Anh Kiệt - 23127077
- Phạm Minh Triết - 23127132
- Nguyễn Hiếu Minh - 23127227
- Trần Quang Phúc - 23127302
- Nguyễn Đức Nhật Minh - 23127421

Agenda & Roadmap





The Distributed Storage Dilemma

When multiple app servers scale horizontally, files written on one node (uploads, logs, assets) are invisible to another. Naive fixes (rsync cron jobs) introduce lag, duplication, and inconsistencies. The requirement: a single shared storage pool accessible concurrently by all compute nodes with minimal latency and no replication delay.

❏ Goal: single source of truth for file I/O across the cluster.

What is NFS?

Network File System (NFS) is a distributed file system protocol that allows a user to access files and directories located on remote computers and treat them as if they were local. It is first developed by **Sun Microsystems** in 1984, and relies heavily on the Open Network Computing Remote Procedure Call, a protocol to invoke a subroutine/procedure on another host on the network.

Common versions of NFS still in use today:

NFSv3 (1995)

Supports 64-bit file sizes and asynchronous writes, improves performance and supports files >2GB

NFSv4 (2000)

Adds stateful operation, integrated security (Kerberos), and better firewall friendliness by using a single TCP/UDP port (2049).

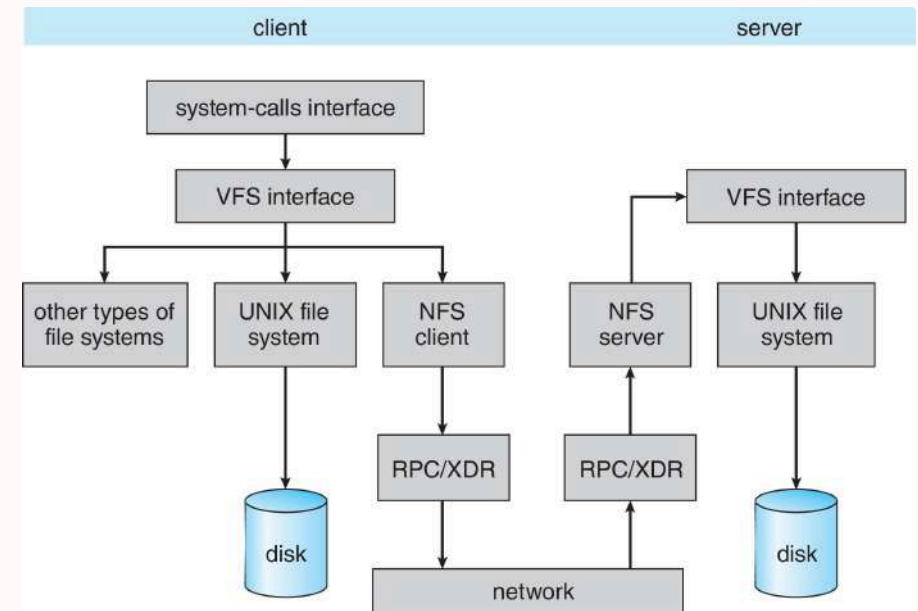
Core Concepts Overview

Server side: the remote hosts with the files.

- **Export:** making a directory on the server available to clients to mount to.
- **Export rules:** Declares which folder to export, allow exports to which hosts (IP), along with export/access options. Located at `/etc/exports`.

Client side: the client host accessing the files on the remote host.

- **Mount:** attaching to a server's **Export**, to access files within the directory.



Understanding Server-Client Dynamics and POSIX Transparency

NFS establishes a clear division of labor between server and client, creating an abstraction layer that makes remote storage behave like local storage.

Server vs. Client Roles

NFS Server

- Manages physical disk storage and defines export policies.
- Listens for incoming client requests via the `nfsd` kernel daemon.
- Enforces access control and permissions for shared directories.

NFS Client

- Accesses remote storage over the network using mount points.
- Treats remote filesystems transparently as local partitions.
- Sends file I/O requests to the server and receives data.

The Abstraction Layer: POSIX Transparency

- Applications on the client node (e.g., web servers, databases) remain unaware that storage resides on a remote host.
- Standard file I/O operations (open, read, write, close) execute directly on the local mount point path.
- No custom application code is required to handle network storage, simplifying development.

Transparent access functions like inserting an SD card into a PC: the OS maps it into the local directory tree, making network distance completely invisible to local applications.

Deep-Dive: Export & /etc/exports Parameters

Fine-Tuning Storage Access, Security, and Integrity Flags

An NFS "export" is the administrative action of exposing a specific server directory path to authorized network clients or subnets, making its contents available for remote mounting and access.

Anatomy of /etc/exports Configuration

The `/etc/exports` file on the NFS server dictates which directories are shared and with whom.

```
/path/to/share client_IP(option1, option2, ...)
```

Key Parameters Breakdown

Access Flags

- rw (Read-Write):** Allows clients full file operations including reading, writing, and modifying files.
- ro (Read-Only):** Restricts clients to only read access, preventing any modifications to the shared directory.

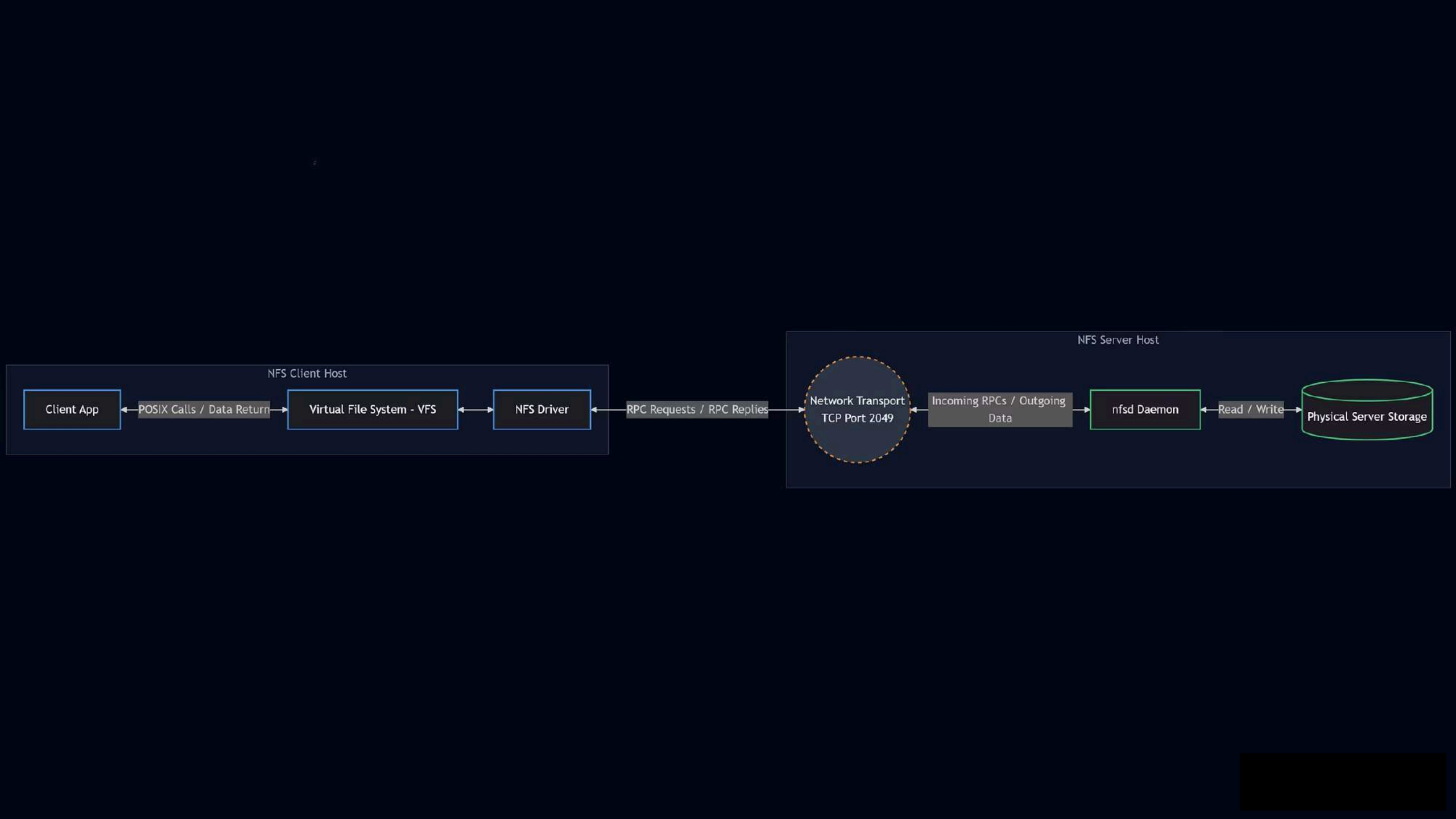
Data Integrity Trade-Off

- sync (Default):** Forces the server to commit changes to physical disk before acknowledging write requests. Ensures maximum safety against data loss during power failures.
- async:** Acknowledges writes as soon as data hits RAM. Offers higher throughput but risks file corruption if the server crashes unexpectedly before data is written to disk.

Performance Optimization

- no_subtree_check:** Disables verification of parent directory trees. This can significantly boost file transfer efficiency and prevent certain rename errors, especially when a client mounts only a subdirectory.





Under the Hood: NFS Architecture (Port 2049)

NFS bridges local I/O to networked storage in the following order

- **VFS** intercepts app I/O on the client and forwards to the **NFS driver**.
- **RPC** transports filesystem operations across the network to the server on TCP/2049.
- **nfsd (NFS's kernel daemon)** receives RPCs and performs raw filesystem writes on the server.
- Once file operation is complete, the server sends the results back if needed.

Use NFSv4 when possible: single predictable port (2049) simplifies firewalling and stateful features.



Server Setup – Step-by-Step

Installation:

```
sudo apt install nfs-kernel-server
```

Create export directory:

```
sudo mkdir -p /shared_directory # or any other directory paths  
sudo chown nobody:nogroup -R /shared_directory # we'll get to the nobody:nogroup later  
sudo chmod 770 -R /shared_directory
```

Configure the export

- Edit in `/etc/exports` (using nano, vim or other editors)

```
sudo exportfs -a # nfs will re-read /etc/exports for new configs  
sudo systemctl restart nfs-kernel-server
```

Server Setup – User (root) Squashing

Root squashing: prevents a remote user with root privileges on a client machine from exercising root privileges on the NFS server.

- With root squashing on, any file owned by root on the client will be owned by `nobody:nogroup` on the NFS server. This user and group are created by default on virtually all Linux distros. NFS assumes these profiles to be there.

Benefits:

- Prevents users from having root privileges on the export, or place malicious payloads owned by root
- Safely handle files when client's UID and GID are not trust-worthy

Best (common) practice:

- Just keep it at `all_squash` most (if not all) of the time. This avoids the problem of a file being owned by a user that is not found on the server host, which causes it to default to a UID instead of a username. Having a consistent ownership is better.

Server Setup – Export configurations

- **Typical format:** `/path client_identifier(option1,option2,...)`
 - Clients can be identified by an **IP**, a **CIDR** range, a **domain name**, or a **wildcard** - though dangerous.
 - Options include:
 - `ro` or `rw`: allows client to either only read or read AND write
 - `sync` or `async`: configure export to write file to disk before send response to client (**sync** - safer), or to respond once file operation finished (**async** - less safe)
 - `no_subtree_check`: disables file-in-subdirectory checks, universally used.
 - `root_squash`: append the `nobody:nogroup` identity trait on file if client attempts to access it as `root`. The opposite of this is `no_root_squash`, which equates client `root` as the server `root` user.
 - `all_squash`: squashes everyone into `nobody:nogroup` regardless of who they are on the client.
- An example:

```
/shared 192.168.1.0/24(rw,sync,no_subtree_check,all_squash) 100.110.2.91(rw,sync,no_subtree_check,all_squash)
```

 Do NOT create multiple configuration lines for the same share. Contradicting rules will be evaluated by the most specific match.

Client Setup: Mounting and Persisting

Virtual File Mapping, Persistency, and Boot Sequence Guards

The **mount operation** is the process of logically binding a remote server's exported directory to a local, empty directory target (known as a **Mount Point**) on the client machine. This allows the client to access files as if they were stored locally.

Volatile Mounting

Using the `mount` command directly binds storage dynamically in memory. The connection is temporary and will be lost upon system reboot.

Persistent Mounting

Configuring entries in `/etc/fstab` automates storage binding on boot. This ensures seamless, continuous application operation across system restarts.

Boot Sequence Guard: The `_netdev` Flag

- ❏ The `_netdev` flag is crucial and mandatory in `/etc/fstab` for NFS mounts. It instructs the Linux initialization process to defer mounting remote NFS volumes until network interfaces are fully online. This prevents critical system hangs or boot freezes if the remote NFS server is temporarily unreachable during startup, ensuring system stability.

Client setup – Mounting and persisting

01

Bootstrap

Install client helpers:

```
sudo apt install nfs-common -y
```

Mount an NFS export:

```
sudo mkdir -p /mnt/nfs_client  
sudo mount 192.168.1.100:/shared /mnt/nfs_client
```

02

Persistence

Add to /etc/fstab for persistent mounts:

```
192.168.1.100:/shared /mnt/nfs_client nfs defaults,_netdev  
0 0
```

 Use IPs or DNS names; prefer fstab with _netdev option for network-aware mounting.

Live Demonstration Roadmap

1

Phase 1 – Baseline

Verify network isolation, identical empty directories on both nodes.

2

Phase 2 – Export & Mount

Run export script on server and perform client mount live.

3

Phase 3 – Real-Time Verification

Create a file on the client; show immediate visibility on the server.

4

Phase 4 – Persistence Test

Add fstab entry, unmount, run mount -a, and confirm recovery after reboot simulation.



Security Deep Dive & Protocol Evolution

- **Unencrypted by default:** the protocols does **one** thing well, so **non-existence security** .

Authentication & Access Control

- **Machine-level trust:** based purely on the client machine's IP address or hostname.
- **Spoofing risks: IP spoofing**

UID/GID Synchronization Requirements

- **The ID alignment problem:** NFS natively assumes the user ID (UID) and group ID (GID) numbers match exactly between the client and server machines.
- **Managing mismatches:** To prevent file ownership chaos, use single centralized /etc/passwd

POSIX ACL Restrictions

- **Client blindness:** Clients cannot read or modify complex POSIX Access Control Lists .
- **Silent enforcement:** The NFS server still enforces the underlying disk ACLs, client receive opaque "permission denied"

Fix?

- **Kerberos :** 3 kerberos security level can be used,
 - krb5 basic secure authentication
 - krb5i - data integrity whilst transit
 - krb5p - privacy (complete encryption)
- **tunnel/VLAN :** to encrypt in transit data

Security Deep Dive & Protocol Evolution : NFSv4

Feature	NFSv3	NFSv4 (NFSv4.1)
Architecture	Stateless: Server maintains no client state	Stateful: Server tracks clients, leases, and locks
Network & Firewall	Multi-port: for multiples services, (NFS, 111 for portmap, 635 for mountd, 4045 for NLM, 4046 for NSM, 4049 for rquota...). idle TCP sessions timeout and cause silent firewall drops	Single-port (2049): Firewall-friendly; lease renewals keep TCP sessions active
Security	Basic: UID/GID-based authentication	Advanced: String-based (user@domain); full Kerberos support (krb5, krb5i, krb5p)
File Locking	Honor: manually editing the lock files	Enforced: lease out locks and server-mandated
Network Partitions	Tolerant: Clients block and wait indefinitely during outages	Sensitive: Locks are revoked if outage exceeds the lease timeout (e.g., 30s)
Server Failure Recovery	Instant: No state reconstruction required	Delayed: Requires a defined "grace period" to rebuild lease/lock state (45s default)

Security Deep Dive & Protocol Evolution : another use case

Feature	NFS (Network File System)	SMB (CIFS)
Origin	Linux / UNIX	Windows
Shared Resources	Files and directories	Files and directories, printers, and network services
Authentication	IP-based / UID (traditionally)	Host local account or
Client Communication	Client-to-Server	Client-to-Server, and Server-mediated Client-to-Client
Best Use Cases	Performance critical Linux env, Cloud Arch, AWS EFS, Kubernetes clusters	Windows corporate networks domain, mixed-OS user to user filesharing

Choose the protocol that matches your ecosystem — NFS for server clusters, Samba for user-facing cross-OS shares.



Production Best Practices & Common Pitfalls

- ❌ Leaving NFS port exposed to public addresses.
- ❌ Unsynchronized numeric UIDs/GIDs between hosts, Production Best Practices
- ✅ Always enforce `sync` rather than `async` in production `/etc/exports` to guarantee file system consistency during unexpected power failures.
- ✅ Use the `_netdev` directive flag inside `/etc/fstab` to instruct the OS to defer mounting until network controllers are fully online.

Conclusion & Key Takeaways

Transparent Efficiency

NFS provides a fast, low-overhead standard for seamless file sharing across native Linux compute.

Automation Reliance

Integrating `/etc/exports` and `/etc/fstab` builds reliable automation for modern clusters.

Modern Standards

consider adopting NFSv4 if you can benefit from the security and concurrency

References & Academic Citations

- IETF RFC 7530 — Network File System (NFS) Version 4 Protocol Specification.
- Linux Reference Manual Pages **exports(5)**, **nfs(5)**, **exportfs(8)**
- Canonical Ubuntu Server Documentation: Network File System Core Configurations.

 **NetApp | Community**

 community.netapp.com



NFSv3 and NFSv4: What's the difference?

NFS has been around for decades as the premier networked, clustered filesystem. If you're a unix/linux user, and you're storing a lot of files,...

 wiki.archlinux.org



NFS – ArchWiki

The NFS FAQ lists file systems well tested and details limitations regarding FAT32.

Questions & Discussion

Thank you for listening! We're now open for your questions and insights.